

Programming & Algorithms

Aula 06: Operadores e expressões relacionais e lógicas; estruturas de seleção simples e compostas

Prof. Ettore Baldini-Neto
ettore.neto@faculdadeimpacta.com.br

Na aula passada

Estudamos :

- Expressões em Python
- Operadores de atribuição e de atribuição composta
- Precedência e Associatividade de operadores de atribuição
- Funções integradas em Python
- Comandos print e input

Aula de hoje

- Operadores e operações relacionais e lógicas
- Precedência e associatividade de tais operadores
- Estruturas de seleção
  - simples
  - compostas

Motivações

Computadores executam o que lhes é instruído e ainda não tem autonomia para tomar decisões próprias!

Com estruturas sequenciais de controle de fluxo conseguimos resolver diversos problemas mais simples.

Ocorre que, muitas vezes temos que escolher se vamos ou não executar determinadas instruções que são baseadas em informações ou dados que só serão conhecidos durante o tempo de execução do programa. Para isso existem as **estruturas de seleção e controle.**

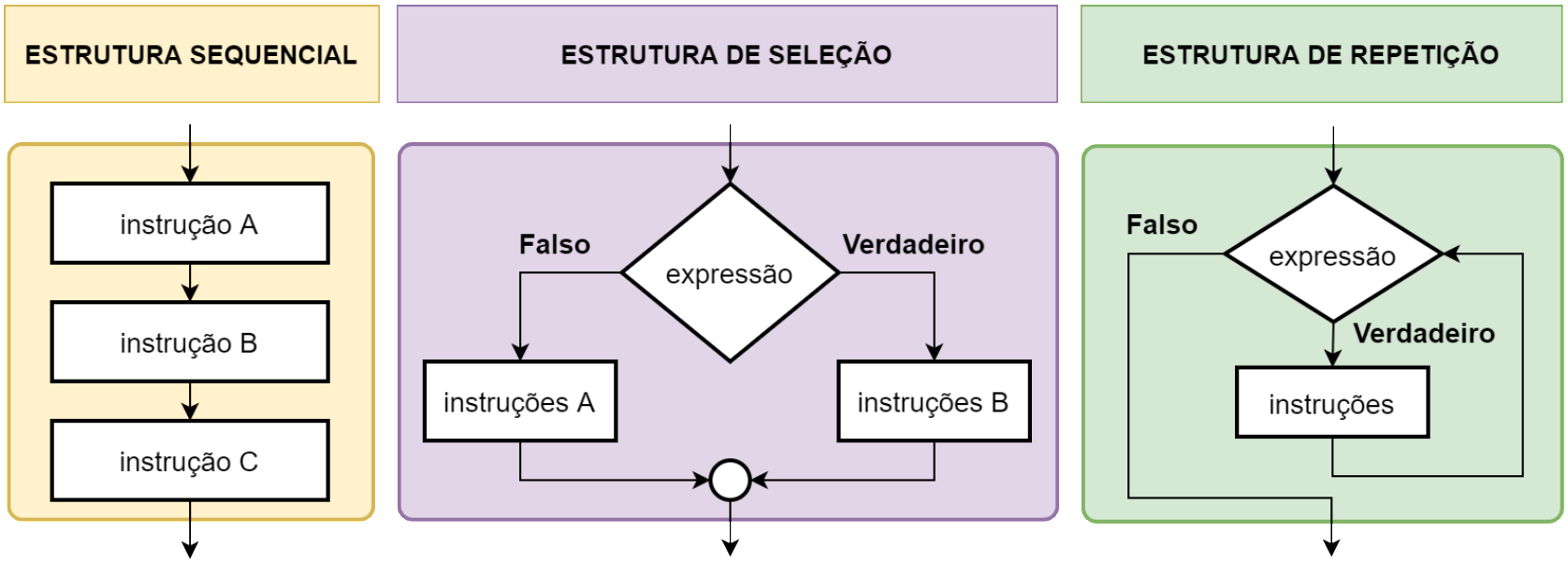
Estruturas de controle

A ordem na qual as instruções de um código são executadas é o **fluxo de execução**. Então, uma estrutura de controle no fluxo de execução é a maneira de controlar a ordem de execução.

Temos três tipos de estruturas de controle de fluxo de execução

- Estrutura sequencial
- Estrutura de seleção (condicional)
- Estrutura de repetição (iterativa), também chamada laço

Estruturas de Controle de Fluxo



Com a estrutura de controle **condicional** pode-se criar programas que decidam **em tempo de execução**, quais instruções executar com base em uma expressão que vai resultar em um **valor lógico.**

Expressões relacionais e lógicas

A avaliação de uma expressão relacional lógica fornece um valor booleano, ou seja, True ou False.

O que é uma expressão relacional?

R. São aquelas em que dois ou mais operandos são comparados para verificar se uma dada relação entre eles é verdadeira ou falsa.

A relação é indicada através de operadores relacionais que são: maior, maior igual, menor, menor igual, diferente, etc

Operadores relacionais

Operador	Descrição	Exemplos

==	igual à	5 ==3 #False
		8 ==8 #True

!=	diferente de	5 != 3 #True 8 !=8 # False
>	maior que	5 > 3 # True 8 > 8 # False
>=	maior ou igual a	5 >= 3 #True 8 >= 8 #True
<	menor que	5 < 3 # False 8 < 8 #False
<=	menor ou igual a	5 <= 3 # False 8 <= 8 # True

Operadores Lógicos

Os operadores lógicos permitem a **combinação** de comparações ou valores booleanos, tornando possível a utilização de expressões mais complexas. Vamos considerar três operadores lógicos

- **not:** operador unário que inverte o valor do operando associado (ou nega o valor do operando associado)
- **and:** operador binário (conexão) que resulta em True apenas se ambos os operandos forem True**.** Do contrário, resulta em False
- **or:** operador binário (disjução) que resulta em True se pelo menos um dos operandos for True**.** Do contrário, resulta em False

Tabelas - Verdade:

a	not a
True	False
False	True

negação

a	b	a and b
True	True	True
True	False	False
False	True	False
False	False	False

conexão

a	b	a or b
True	True	True
True	False	True
False	True	True
False	False	False

disjunção

Alguns exemplos

Exemplo 1

```
>>> a = 1
>>> b = 5
>>> c = 2
>>> d = 1
>>> a == b
False
```

```
>>> a < b
True
>>> a == d
True
>>> b >= a
True
>>> c <= b
True
>>> d != a
False
>>> d != b
True
```

Atenção!

```
>>> d >= a
True
>>> a <= d
True
```

Alguns exemplos de codificações

Variáveis do tipo lógica podem ser utilizadas para armazenar o resultado de expressões e comparações.

```
>>> nota = 8
>>> media = 7
>>> aprovado = nota > media
>>> aprovado
True
```

Se um expressão contiver operações aritméticas, estas devem ser calculadas **antes** que os operadores relacionais sejam avaliados.

```
>>> b > a + c + d
True
```

lembrando que

```
>>> a = 1
>>> b = 5
>>> c = 2
>>> d = 1
```

Alguns exemplos de codificações

```
>>> idade = 19
>>> pode_tirar_cnh = idade >= 18
>>> print(pode_tirar_cnh)
True
>>>
```

```
>>> idade = 19
>>> pode_tirar_cnh = idade >= 18
>>> aprovado_no_detran = True
>>> pode_dirigir = pode_tirar_cnh and aprovado_no_detran
>>> print(pode_dirigir)
True
>>>
```

Alguns exemplos de codificações

Atenção: A avaliação de string para bool não faz nenhuma interpretação, logo uma string vazia vai retornar False enquanto qualquer outra possibilidade vai retornar True.

Então **não use** a função bool neste caso.

Exemplo do que não fazer: ~~aprovado_no_detran = bool(input( )). X~~

```
idade = int(input('Entre com sua idade: '))
aprovado_no_detran = input('Você passou na prova? (s/n)') == 's'

pode_tirar_cnh = idade >= 18
pode_dirigir = pode_tirar_cnh and aprovado_no_detran
print('Você pode dirigir?', pode_dirigir)
```

Precedência e associatividade

Tabela 4.4: Precedência e associatividade dos operadores relacionais e lógicos.

Operador	Descrição	Associatividade
`==`, `!=`, `>`, `>=`, `<`, `<=`	Operadores relacionais.	Não associativos
`not`	Negação lógica.	À esquerda
`and`	E lógico.	
`or`	OU lógico.	

Python permite o encadeamento de operadores relacionais. O interpretador traduz a expressão como um encadeamento de expressões binárias repetindo os operadores relacionais e unindo-os.

Precedência e associatividade

Vejamos como funciona com um exemplo:

1. 3 < 10 < 23 → 3 < 10 and 10 < 23
2. 3 < 10 and 10 < 23 → True and 10 < 23
3. True and 10 < 23 →True and True
4. True and True → True

Precedência e associatividade

Tabela 4.5: Prioridade dos operadores aritméticos, relacionais e lógicos.

Ordem de resolução	Operador	Descrição	Associatividade
1°	`**`	Exponenciação.	À direita
2°	`+`, `-` (unários)	Identidade e negação.	À esquerda
3°	`*`, `/`, `//`, `%`	Multiplicação, divisão real, divisão inteira e resto da divisão.	
4°	`+`, `-` (binários)	Adição e subtração.	
5°	`==`, `!=`, `>`, `>=`, `<`, `<=`	Operadores relacionais.	Não associativos
6°	`not`	Negação lógica.	À esquerda
7°	`and`	E lógico.	
8°	`or`	OU lógico.	

Expressões equivalentes e complementares

Duas expressões são **equivalentes** quando têm **exatamente o mesmo significado,** mesmo se escritas de formas distintas.

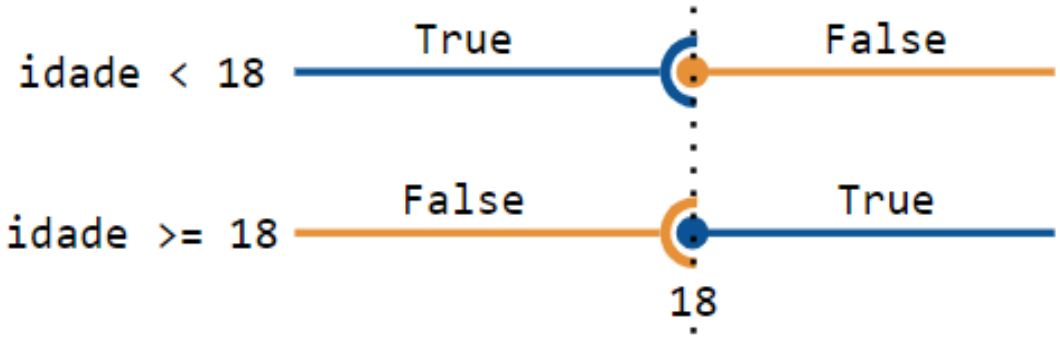
Exemplo

```
>>> idade = 51
>>> idade >= 18
True
>>> 18 <= idade
True
>>>
```

Expressões equivalentes e complementares

Duas expressões são **complementares** quando uma é a negação da outra, ou seja, juntas cobrem todo o espaço possível de valores envolvidos **sem sobreposição**.

Exemplo: idade < 18 é complementar à idade >=18.



Expressões equivalentes e complementares

Exemplos de expressões complementares:

```
>>>
>>> 'Maria' != 'Megan'
True
>>> 'Maria ' == 'Megan'
False
>>>
>>> not('Maria' == 'Megan')
True
>>> not('Maria'!= 'Megan')
False
>>>
```

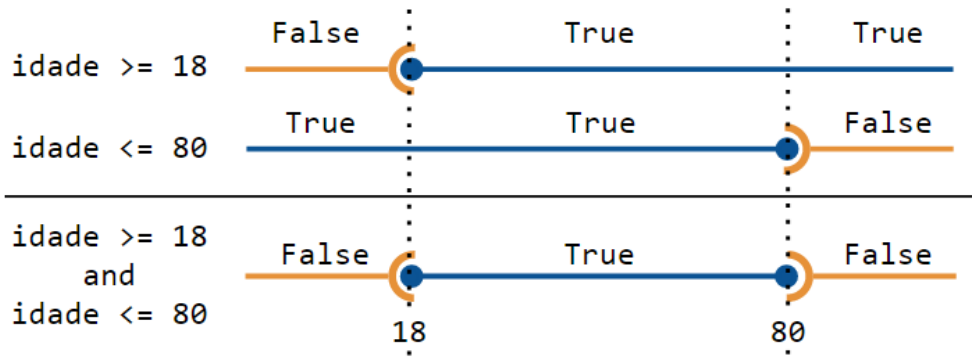
Expressões com operadores relacionais e lógicos

Exemplo: Pessoas acima de 80 anos não podem ter cnh

```
>
> pode_tirar_cnh = idade >= 18 and idade <= 80
>
```

Expressões equivalentes:

```
>>> 18 <= idade and idade <= 80
True
>>> 18 <= idade <= 80
True
>>>
```



Expressões com operadores relacionais e lógicos

A expressão complementar, ou seja, aquela que resulta em True para aqueles que **não podem tirar a cnh** é:

idade < 18 or idade > 80

Resumindo: idade >= 18 and idade <= 80

expressões complementares:

1. not(idade >= 18 and idade <= 80)
2. not (idade >=18) or not (idade <= 80)
3. idade < 18 or idade > 80

Expressões com operadores relacionais e lógicos

Conclusão: Ao negar uma expressão inicial obtemos uma complementar. O operador lógico **and** é substituído por **or** e os operadores relacionais >= e <= são substituídos por < e >, respectivamente

Avaliação de curto circuito

Operadores lógicos com avaliação de curto circuito estabelecem que seu segundo operando **só** será avaliado **caso** a avaliação do primeiro **não seja suficiente para concluir o valor da expressão completa.**

- **and:** o segundo operando só será avaliado caso o primeiro resulte em **True** pois uma expressão com and o resultado só pode ser antecipado se o primeiro operando resultar em **False**
- **or:** o segundo operando só será avaliado caso o primeiro resulte em **False** pois em uma expressão com or, o resultado só pode ser antecipado se o primeiro operando resultar em **True**

Avaliação de curto circuito

a	NÃO a
True	False
False	True

a	b	a E b
True	True	True
True	False	False
False	True	False
False	False	False

a	b	a OU b
True	True	True
True	False	True
False	True	True
False	False	False

Avaliação de curto circuito

```
# Programa que recebe 2 números naturais e exibe uma mensagem
# indicando se o primeiro é divisível pelo segundo

a = int(input('digite o primeiro número: '))
b = int(input('digite o segundo número: '))
print(f'{a} é divisível por {b}: {b != 0 and a % b == 0}')
```

Sem a avaliação de CC, se o usuário inserisse zero para b, o programa geraria um erro pela tentativa de divisão por zero.

Note que a % b == 0 **só será avaliada** se b !=0 resultar True.

Avaliação de curto circuito

```
a = int(input('digite o primeiro número: '))
b = int(input('digite o segundo número: '))
print(f'{a} não é divisível por {b} : {b == 0 or a % b != 0}')
```

```
a = int(input('digite o primeiro número: '))
b = int(input('digite o segundo número: '))
print(f'{a} não é divisível por {b} : {not(b != 0 and a % b == 0)}')
```

Sem a avaliação de CC, se o usuário inserisse zero para b, o programa geraria um erro pela tentativa de divisão por zero. Como a % b != 0 **só será avaliada após** b ==0 resultar False, o erro não ocorrerá.

Estruturas de seleção (condicionais)

São estruturas utilizadas para selecionar se um trecho de código será executado baseado na avaliação de uma expressão. Ou seja, a execução do trecho de código está condicionada ao resultado da expressão.

Tipos de estruturas condicionais

- **Simples:** quando o bloco de código for executado apenas quando a condição resultar em True.
- **Composta**: quando há dois blocos de código condicionados e um deles só será executado apenas quando a condição resultar True e o outro será executado apenas quando a condição resultar False.

Ou seja, havendo a execução de um, o outro será ignorado.

Estruturas de seleção

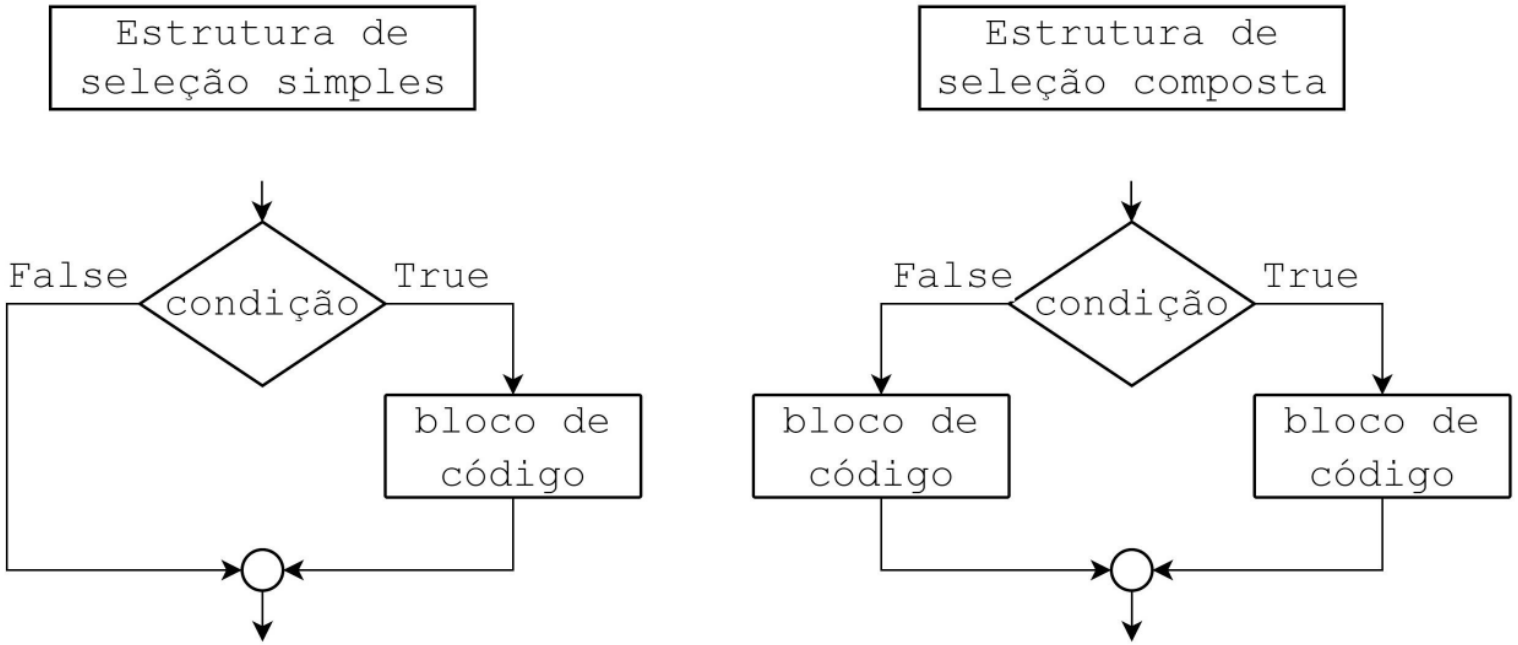


Figura 4.5: Fluxograma das estruturas de seleção simples e composta.
Fonte: Elaborado pelo autor.

Estruturas de seleção simples

```
if < condição >:
___ <bloco de código>
```

- O bloco de código é introduzido pelos dois pontos **:**
- Instruções pertencentes ao bloco estão no mesmo nível de indentação entre si e **indentadas de 4 espaços relativamente à instrução que introduz o bloco.**

Estruturas de seleção simples

```
idade = int(input('Qual sua idade? '))

if idade >= 18:
    #Bloco executado quando a condição é verdadeira
    print('Você pode ter uma CNH.')
    print('Desejamos boa sorte!')

print('Fim')
```

Estruturas de seleção composta

```
if < condição>:
    <bloco de código 1>
else:
    <bloco de código 2>
```

Após o comando else não é preciso (nem permitido) colocar uma segunda condição. Quando a condição if resultar True, o bloco 1 será executado e o bloco 2 ignorado. Quando a condição if resultar False ocorrerá o oposto.

Note bem: else é complemento do if, por isso **nunca deve ser escrito isoladamente**.

Estruturas de seleção compostas

```
idade = int(input('Qual sua idade? '))

if idade >= 18:
    print('Você pode ter uma CNH.')
    print('Desejamos boa sorte!')
else:
    print('Você ainda não completou 18 anos, ', end = '')
    print('portanto não pode ter uma CNH. ')

print('Fim.')
```

```
idade = int(input('Entre com sua idade: '))
aprovado_no_detran = input('Você passou na prova? (s/n)') == 's'

if 18 <= idade <= 80 and aprovado_no_detran:
    print('Você pode dirigir')
else:
    print(f'Você precisa obedecer às condições estabelecidas')

print('Fim!')
```

Estruturas de seleção compostas

Atenção: Nem todo if deve ser acompanhado por um else.

Quando existem blocos de instruções mutuamente exclusivas devemos usar seleção simples!

Estruturas de seleção compostas.

```
#Crie um programa que receba como entrada o valor de um produto e
# a quantidade comprada. O programa dará 10% de desconto para compras
#com total maior ou igual a RS100,00

valor = float(input('Valor: '))
quantidade = int(input('Quantidade: '))
total = valor * quantidade

if total >= 100.00:
    total = total * 0.9
else:
    total = total

print('Total:', total)
```

O else é totalmente desnecessário.

Comente este bloco e rode de novo o programa

Caso queiram imprimir com 2 casas decimais, usem: print(f' Total = {total:.2f}')

Exercício

Equação de 2 grau: $ax^2 + bx + c = 0$

Soluções:

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

O trecho intermediário **$b^2 - 4ac$** também é chamado de discriminante ou delta (Δ) da equação.

Quando $\Delta = 0$ raiz é x = -b/2a

Exercício

- Faça um programa em Python que peça para o usuário digitar os valores dos coeficientes a, b, e c da equação do 2 grau e que calcule:

a. O valor do Delta

b. Se Delta > 0 imprime "A equação tem 2 raizes reais"

c. Se Delta = 0 imprime "A equação tem uma única raiz real"

d. Se Delta < 0 imprime "A equação não possui raizes reais" e encerra o programa.

e. Nos casos b e c o programa deve calcular as 2 raizes de equação e imprimir na tela com três casas decimais.

FIM